



University Mohamed Boudiaf of M'sila

Department of Computer Science

Master 1 – Artificial Intelligence

Module: Machine Learning & Data Mining (FDTA)

Laboratory Report – TP N°4

Decision Tree Implementation with R

Prepared by: Younes Barkat

Supervised by: Dr. Taher MEHANNI

Academic Year: 2025/2026

Step 1: Installing the 'caret' Package

```
install.packages("caret", lib = "C:/Users/Younes/Desktop/TP_FDDTA/TP4/Caret")
```

```
> install.packages("caret", lib = "C:/Users/Younes/Desktop/TP_FDDTA/TP4/Caret")
WARNING: Rtools is required to build R packages but is not currently installed. Please download and install the appropriate version of Rtools before proceeding:
https://cran.rstudio.com/bin/windows/Rtools/
trying URL 'https://cran.rstudio.com/bin/windows/contrib/4.5/caret_7.0-1.zip'
Content type 'application/zip' length 3604745 bytes (3.4 MB)
downloaded 3.4 MB
package 'caret' successfully unpacked and MD5 sums checked
The downloaded binary packages are in
C:\Users\Younes\AppData\Local\Temp\RtmpKavhyD\downloaded_packages
```

The caret package is installed from CRAN into a user-specified local library directory. The console confirms a successful download of version 7.0-1 (3.4 MB) with MD5 integrity verification.

Step 2: Loading the 'caret' Library

```
library(caret)
```

```
> library(caret)
Loading required package: ggplot2
Loading required package: lattice
Warning messages:
1: package 'caret' was built under R version 4.5.3
2: package 'ggplot2' was built under R version 4.5.3
```

Loading caret automatically imports its dependencies ggplot2 and lattice. The version compatibility warnings indicate the package was compiled under R 4.5.3, which is consistent with the current R environment.

Step 3: Installing the 'rpart.plot' Package

```
install.packages("rpart.plot")
```

```
> install.packages("rpart.plot")
WARNING: Rtools is required to build R packages but is not currently installed. Please download and install the appropriate version of Rtools before proceeding:
https://cran.rstudio.com/bin/windows/Rtools/
Installing package into 'C:/Users/Younes/AppData/Local/R/win-library/4.5'
(as 'lib' is unspecified)
trying URL 'https://cran.rstudio.com/bin/windows/contrib/4.5/rpart.plot_3.1.4.zip'
Content type 'application/zip' length 1018701 bytes (994 KB)
downloaded 994 KB
package 'rpart.plot' successfully unpacked and MD5 sums checked
The downloaded binary packages are in
C:\Users\Younes\AppData\Local\Temp\RtmpKavhyD\downloaded_packages
```

The rpart.plot package, responsible for visualizing decision trees, is fetched from CRAN (994 KB). It is installed alongside its core dependency rpart, enabling graphical rendering of tree structures.

Step 4: Loading the 'rpart.plot' Library

```
library(rpart.plot)
```

```
> library(rpart.plot)
```

```
Loading required package: rpart
```

```
Warning message:
```

```
package 'rpart.plot' was built under R version 4.5.3
```

Loading `rpart.plot` triggers the import of its underlying `rpart` engine. The warning confirms the package was built under R 4.5.3 and will function correctly in this session.

Step 5: Importing the Car Evaluation Dataset

```
data_url <- c("https://archive.ics.uci.edu/ml/machine-learning-  
databases/car/car.data")  
download.file(url = data_url, destfile = "car.data")  
car_df <- read.csv("car.data", sep = ',', header = FALSE)
```

```
> data_url <- c("https://archive.ics.uci.edu/ml/machine-learning-databases/car/car.data")
```

Values

data_url	"https://archive.ics.uci.edu/ml/machin...
----------	---

```
> download.file(url = data_url, destfile = "car.data")  
trying URL 'https://archive.ics.uci.edu/ml/machine-learning-databases/car/car.data'  
downloaded 50 KB
```

```
> car_df <- read.csv("car.data", sep = ',', header = FALSE)
```

The UCI Car Evaluation dataset is downloaded directly from its repository and stored locally as 'car.data'. It is then loaded into a data frame without column headers, yielding 1728 observations across 7 variables (V1–V7).

Step 6: Exploring the Dataset Structure

```
str(car_df)  
head(car_df)
```

	V1	V2	V3	V4	V5	V6	V7
1	vhigh	vhigh	2	2	small	low	unacc
2	vhigh	vhigh	2	2	small	med	unacc
3	vhigh	vhigh	2	2	small	high	unacc
4	vhigh	vhigh	2	2	med	low	unacc
5	vhigh	vhigh	2	2	med	med	unacc
6	vhigh	vhigh	2	2	med	high	unacc
7	vhigh	vhigh	2	2	big	low	unacc
8	vhigh	vhigh	2	2	big	med	unacc
9	vhigh	vhigh	2	2	big	high	unacc
10	vhigh	vhigh	2	4	small	low	unacc

```
> str(car_df)
'data.frame': 1728 obs. of 7 variables:
 $ V1: chr "vhigh" "vhigh" "vhigh" "vhigh" ...
 $ V2: chr "vhigh" "vhigh" "vhigh" "vhigh" ...
 $ V3: chr "2" "2" "2" "2" ...
 $ V4: chr "2" "2" "2" "2" ...
 $ V5: chr "small" "small" "small" "med" ...
 $ V6: chr "low" "med" "high" "low" ...
 $ V7: chr "unacc" "unacc" "unacc" "unacc" ...
```

```
> head(car_df)
  V1 V2 V3 V4 V5 V6 V7
1 vhigh vhigh 2 2 small low unacc
2 vhigh vhigh 2 2 small med unacc
3 vhigh vhigh 2 2 small high unacc
4 vhigh vhigh 2 2 med low unacc
5 vhigh vhigh 2 2 med med unacc
6 vhigh vhigh 2 2 med high unacc
```

The `str()` function confirms the data frame consists of 1728 records with 7 character-type columns. The `head()` preview shows that the first several observations are all classified as 'unacc', reflecting a dominance of the unacceptable class in the dataset.

Step 7: Partitioning the Dataset (70% Train / 30% Test)

```
set.seed(3033)
intrain <- createDataPartition(y = car_df$V7, p = 0.7, list = FALSE)
training <- car_df[intrain,]
testing <- car_df[-intrain,]
```

```
> set.seed(3033)
```

```
> intrain <- createDataPartition(y = car_df$V7, p= 0.7, list = FALSE)
```

	Resample1
1	1
2	2
3	3
4	4
5	5
6	7
7	8
8	9
9	10
10	11

```
> training <- car_df[intrain,]
```

	V1	V2	V3	V4	V5	V6	V7
1	vhigh	vhigh	2	2	small	low	unacc
2	vhigh	vhigh	2	2	small	med	unacc
3	vhigh	vhigh	2	2	small	high	unacc
4	vhigh	vhigh	2	2	med	low	unacc
5	vhigh	vhigh	2	2	med	med	unacc
7	vhigh	vhigh	2	2	big	low	unacc
8	vhigh	vhigh	2	2	big	med	unacc
9	vhigh	vhigh	2	2	big	high	unacc
10	vhigh	vhigh	2	4	small	low	unacc
11	vhigh	vhigh	2	4	small	med	unacc

```
> testing <- car_df[-intrain,]
```

The seed ensures reproducibility. createDataPartition performs stratified sampling on the target variable V7 with a 70/30 ratio, producing a training set of 1212 samples and a test set of 516 samples, preserving class distribution across both subsets.

Step 8: Checking for Missing Values

```
anyNA(car_df)
```

```
> anyNA(car_df)
[1] FALSE
```

The anyNA() function scans all columns for NA entries and returns FALSE, confirming that the dataset is complete with no missing values. This eliminates the need for imputation prior to model training.

Step 9: Configuring Training Control (Repeated Cross-Validation)

```
trctrl <- trainControl(method = "repeatedcv", number = 10, repeats = 3)
```

```
> trctrl <- trainControl(method = "repeatedcv", number = 10, repeats = 3)
```

Name	Type	Value
trctrl	list [27]	List of length 27
method	character [1]	'repeatedcv'
number	double [1]	10
repeats	double [1]	3
search	character [1]	'grid'
p	double [1]	0.75
initialWindow	NULL	Pairlist of length 0
horizon	double [1]	1
fixedWindow	logical [1]	TRUE
skip	double [1]	0
verboseIter	logical [1]	FALSE
returnData	logical [1]	TRUE
returnResamp	character [1]	'final'
savePredictions	logical [1]	FALSE
classProbs	logical [1]	FALSE
summaryFunction	function	function(data, lev = , model =) { ... }
selectionFunction	character [1]	'best'
preProcOptions	list [6]	List of length 6
sampling	NULL	Pairlist of length 0
index	NULL	Pairlist of length 0
indexOut	NULL	Pairlist of length 0
indexFinal	NULL	Pairlist of length 0
timingSamps	double [1]	0
predictionBounds	logical [2]	FALSE FALSE

The trainControl object specifies 10-fold cross-validation repeated 3 times, yielding 30 resampling iterations in total. This robust evaluation strategy reduces variance in performance estimates by averaging results across multiple independent folds.

Step 10: Building the Decision Tree (Information Gain)

```
set.seed(3333)
dtree_fit <- train(V7 ~., data = training, method = "rpart",
  parms = list(split = "information"),
  trControl = trctrl,
  tuneLength = 10)
```

```
> set.seed(3333)
```

```
> dtree_fit <- train(V7 ~., data = training, method = "rpart",
+                   parms = list(split = "information"),
+                   trControl=trctrl,
+                   tuneLength = 10)
```

Name	Type	Value
dtree_fit	list [25] (S3: train, train.formula)	List of length 25
method	character [1]	'rpart'
modelInfo	list [15]	List of length 15
modelType	character [1]	'Classification'
results	list [10 x 5] (S3: data.frame)	A data.frame with 10 rows and 5 columns
pred	NULL	Pairlist of length 0
bestTune	list [1 x 1] (S3: data.frame)	A data.frame with 1 row and 1 column
call	language	train.formula(form = V7 ~ ., data = training, method = "rpart", parms = lis ...
dots	list [1]	List of length 1
metric	character [1]	'Accuracy'
control	list [27]	List of length 27
finalModel	list [19] (S3: rpart)	List of length 19
preProcess	NULL	Pairlist of length 0
trainingData	list [1211 x 7] (S3: data.frame)	A data.frame with 1211 rows and 7 columns
pType	list [0 x 6] (S3: data.frame)	A data.frame with 0 rows and 6 columns
resample	list [30 x 3] (S3: data.frame)	A data.frame with 30 rows and 3 columns
resampledCM	list [300 x 18] (S3: data.frame)	A data.frame with 300 rows and 18 columns
perfNames	character [2]	'Accuracy' 'Kappa'
maximize	logical [1]	TRUE
yLimits	NULL	Pairlist of length 0
times	list [3]	List of length 3
levels	character [4]	'acc' 'good' 'unacc' 'vgood'
terms	formula	V7 ~ V1 + V2 + V3 + V4 + V5 + V6
coefnames	character [15]	'V1low' 'V1med' 'V1vhigh' 'V2low' 'V2med' 'V2vhigh' ...

The CART classifier is trained using information gain as the splitting criterion, evaluated over 10 complexity parameter (cp) values. The optimal model is selected at cp = 0.005494505, achieving a cross-validated accuracy of approximately 87.2% and Kappa of 0.729.

```
> dtree_fit
CART

1211 samples
 6 predictor
 4 classes: 'acc', 'good', 'unacc', 'vgood'

No pre-processing
Resampling: Cross-Validated (10 fold, repeated 3 times)
Summary of sample sizes: 1090, 1092, 1089, 1089, 1089, 1090, ...
Resampling results across tuning parameters:
```

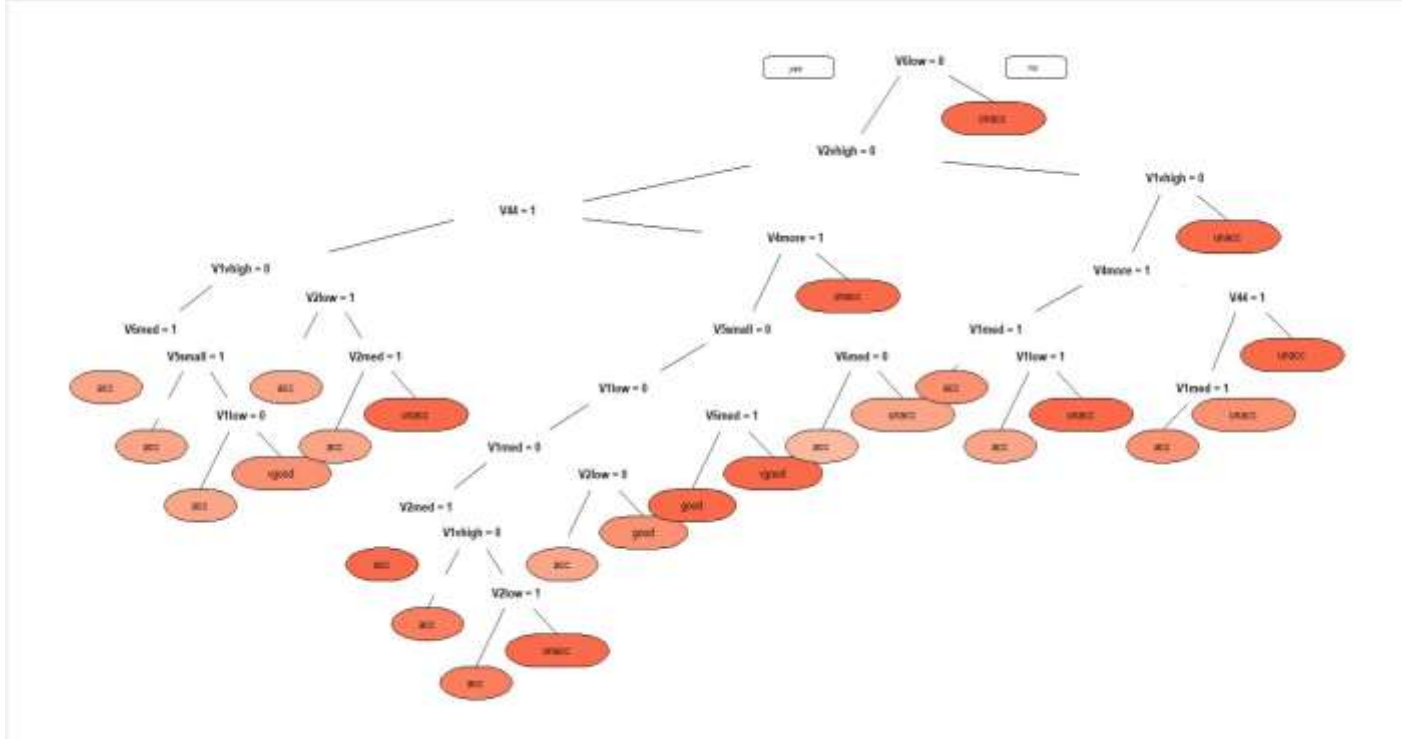
cp	Accuracy	Kappa
0.005494505	0.8717704	0.7290692
0.007692308	0.8621260	0.7055343
0.008241758	0.8610377	0.7015747
0.010989011	0.8538908	0.6820370
0.016483516	0.8219632	0.6100808
0.018543956	0.8115194	0.5848871
0.019230769	0.8115194	0.5848871
0.024725275	0.8046066	0.5785861
0.060439560	0.7863800	0.5568316
0.070054945	0.7643436	0.4594143

Accuracy was used to select the optimal model using the largest value.
The final value used for the model was cp = 0.005494505.

Step 11: Visualizing the Decision Tree

```
prp(dtree_fit$finalModel, box.palette = "Reds", tweak = 1.2)
```

```
> prp(dtree_fit$finalModel, box.palette = "Reds", tweak = 1.2)
```



The `prp()` function renders the final decision tree with a red color palette. The root node splits on V6 (safety), as it carries the highest information gain. Subsequent internal nodes further partition the data using purchase cost (V1), maintenance cost (V2), and passenger capacity (V4).

Step 12: Predicting a Single Test Instance

```
testing[1,]  
predict(dtree_fit, newdata = testing[1,])
```

```
> testing[1,]  
      V1    V2 V3 V4  V5   V6   V7  
6 vhigh vhigh  2  2 med high unacc  
> predict(dtree_fit, newdata = testing[1,])  
[1] unacc  
Levels: acc good unacc vgood
```

The first test record (V1=vhigh, V2=vhigh, V3=2, V4=2, V5=med, V6=high) is correctly classified as 'unacc' by the model. This confirms the tree's ability to assign class labels for individual instances based on learned decision boundaries.

Step 13: Predicting All Test Instances

```
test_pred <- predict(dtree_fit, newdata = testing)  
confusionMatrix(test_pred, testing$V7)
```

```
> test_pred <- predict(dtree_fit, newdata = testing)
```

test_pred	Factor w/ 4 levels "acc","good","unacc",...: 3 3 3...
-----------	---

```

> test_pred
[1] unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc
[15] unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc
[29] unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc
[43] unacc unacc unacc acc unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc
[57] unacc acc acc unacc acc acc unacc unacc unacc acc acc unacc unacc unacc unacc unacc
[71] unacc unacc acc acc acc acc unacc unacc unacc unacc acc unacc acc acc unacc unacc
[85] acc acc acc acc unacc unacc unacc unacc unacc unacc unacc unacc acc acc unacc unacc
[99] acc acc unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc acc unacc unacc
[113] unacc unacc acc unacc unacc unacc unacc unacc unacc unacc unacc acc acc unacc acc acc
[127] acc unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc
[141] unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc
[155] unacc unacc unacc acc unacc unacc unacc unacc unacc acc acc unacc unacc unacc unacc
[169] unacc unacc unacc acc unacc acc unacc unacc unacc unacc unacc acc acc acc unacc unacc
[183] acc acc unacc unacc unacc unacc unacc unacc unacc unacc unacc acc unacc unacc unacc
[197] unacc unacc unacc acc acc acc acc acc unacc unacc acc acc acc acc acc
[211] unacc unacc acc unacc unacc unacc acc acc unacc unacc unacc unacc acc unacc acc
[225] acc unacc unacc unacc acc unacc unacc unacc acc unacc unacc acc unacc unacc
[239] unacc acc unacc unacc acc unacc unacc unacc acc unacc unacc acc unacc acc
[253] acc unacc unacc unacc acc acc acc acc acc acc acc unacc unacc unacc unacc unacc
[267] acc unacc acc acc acc acc acc unacc unacc unacc acc unacc unacc unacc acc
[281] acc unacc unacc unacc acc unacc acc acc unacc unacc unacc unacc unacc unacc unacc
[295] unacc acc acc acc acc acc unacc acc unacc unacc unacc acc unacc unacc
[309] unacc acc acc acc unacc unacc unacc unacc acc unacc acc acc acc acc unacc
[323] acc unacc unacc unacc acc unacc acc acc acc unacc acc unacc unacc unacc unacc
[337] unacc acc unacc acc acc unacc unacc unacc unacc unacc unacc unacc unacc unacc unacc
[351] unacc unacc acc acc unacc unacc acc acc acc unacc good good unacc unacc acc acc
[365] unacc unacc unacc acc acc unacc good good unacc unacc unacc unacc unacc acc acc
[379] good unacc unacc unacc unacc unacc unacc unacc acc acc unacc acc unacc unacc unacc
[393] good unacc unacc unacc unacc unacc unacc unacc unacc acc acc unacc acc unacc unacc
[407] unacc unacc unacc unacc unacc unacc unacc unacc unacc acc unacc acc unacc unacc
[421] unacc unacc unacc unacc unacc acc acc acc acc unacc unacc unacc vgood unacc
[435] vgood unacc unacc unacc unacc acc unacc good vgood unacc unacc unacc vgood unacc
[449] unacc unacc unacc unacc acc vgood unacc good vgood acc unacc unacc unacc acc
[463] unacc acc vgood unacc vgood unacc unacc unacc unacc acc vgood unacc unacc unacc
[477] unacc good unacc unacc acc acc unacc acc vgood unacc vgood unacc unacc unacc
[491] unacc acc unacc unacc unacc unacc unacc acc unacc vgood acc unacc good unacc

```

The model is applied to all 516 test records. The resulting confusion matrix reports an overall accuracy of 83.72% and a Kappa of 0.6703. The 'unacc' class achieves the highest sensitivity (89.83%) and specificity (96.91%), while the 'good' class shows lower recall due to its limited representation in the dataset.